Two targeted improvements to TQP-Inmem

Lyon | Starter research report | September 28, 2026

Tasks: Aggregation Performance and Data Materialization Overhead.
Verified code commit: 045af6abf0f39fcc0ee8bd861b61005249842566

1. Motivation and hypotheses

This project investigates redundant tensor work in two components of TQP-Inmem. The first hypothesis is that multiple aggregate expressions sharing the same grouping can reuse subgroup indices. The second is that projection can preserve result types instead of converting to float64 and then back. Independent, default-off switches permit controlled comparisons. The goal is a measured proof of concept, not a claim of universally faster query execution.

2. Aggregation: reuse subgroup assignments

In operators/aggregate.py, the existing two-stage reduction activates when rows/groups is at least 65,536. For each reduction, scatter_reduce_2steps constructs subgroup_id with arange and remainder, then computes flat_index = inverted_index * num_subgroups + subgroup_id. Q1 repeats this preparation across sums and averages, including average denominators.

The change passes an optional aggregation_cache through recursive evaluate calls. A fresh dictionary is created for each aggregation invocation and stores flat indices by subgroup count. It is not shared across different grouping inputs or queries. TQP_AGG_REUSE=1 enables it; zero preserves the original behavior. The reduction formulas and grouping algorithm are unchanged. The expected benefit is strongest when several reductions share an active two-stage path.

3. Materialization: preserve projection result types

In operators/project.py, the original path records a result dtype, casts the result to float64, copies it into GPU pool storage, then casts it back. TQP_PROJECT_NATIVE=1 skips the forced float64 conversion. malloc_like already sizes and views storage according to the actual tensor dtype. The final same-dtype conversion therefore need not change types. The pool copy is retained: removing it would require a broader ownership/lifetime analysis.

An initial test failed on a non-contiguous int8 substring result. The old type conversion had incidentally made a compact copy. The allocator attempted view(-1).numel() only to count elements, which fails on that slice. Replacing it with numel() avoids flattening; the existing assignment copies the strided source into compact destination storage. The failed attempt and this correction are part of the result.

4. Hardware and common setup

NVIDIA RTX 5080, 16,303 MiB reported memory; NVIDIA driver 590.48.01. CPU: Intel Xeon w5-2455X, 12 cores / 24 threads. Python 3.12.14, PyTorch 2.14.0+cu130, CUDA runtime 13.0. The driver reports CUDA support 13.1; that is not the runtime version. All TPC-H measurements use SF1. Data generation used CPU PyTorch 2.6.0 separately.

The GPU pool was reduced from the upstream 40 GB to 8 GB (decimal) to fit this card. All modes use the same pool and allocator sizing fix. Those setup changes are not counted as an optimization speedup. The original cu126 installation lacked kernels for this GPU; cu130 GPU arithmetic was verified before experiments.

5. Runtime results and interpretation

Each runtime configuration used five separate processes, each with two whole-suite SF1 warmups before the measured queries. Profiling was disabled. The CSV timer subtracts scan and output time: it is not total ingestion or process latency. The final batch rotated mode order over rounds. Values below are medians in milliseconds.

Query	Neither	Materialization	Aggregation	Both
Q1	16.842	16.796	12.930	11.133
Q13	28.507	27.563	27.073	25.735
Q18	20.988	18.211	19.688	16.076
Q22	10.335	6.735	10.215	6.745

An earlier alternating batch found Q1 decreased from 14.776 to 10.933 ms with reuse (26.0% less time); Q13 and Q18 changed by less than 1%. A separate projection comparison found Q22 decreased from 6.328 to 6.127 ms (3.2%), while Q1 was essentially unchanged. These batches should not be pooled as identical conditions.

The final batch was noisy: baseline Q22 ranged from 6.953 to 11.570 ms. Its large apparent median improvement is not a reliable precise effect size. Shared-machine utilization was not systematically captured, so the cause of variation is unknown. Diagnostic Q1 profiles showed aggregation multiplication calls dropping from 18 to 8 and addition calls from 12 to 2, supporting the eliminated-work explanation. Profile timings are not used for speedup claims.

6. Scaling and peak memory

A synthetic benchmark computes sum/average twice on resident GPU inputs, with three warmups and eight timed invocations per mode, alternating order and synchronizing around host-clock timing. Input preparation and direct reference checks are outside timing. These are operator tests, not larger TPC-H scale factors.

Rows	Groups	Two-stage	Original ms	Reuse ms
65,536	4	No	0.289535	0.288719
65,536	64	No	0.233086	0.232907
262,144	4	Yes	0.271941	0.183646
262,144	64	No	0.190565	0.184929
1,048,576	4	Yes	0.421779	0.257982
1,048,576	64	No	0.263736	0.269573

The active two-stage cases used 32.5% and 38.8% less median time. Inactive cases showed small, mixed differences, with individual outliers. This supports a rows-per-group mechanism rather than universal improvement as data size increases.

Across three repetitions, Q1 peak allocated bytes were 8,493,014,528 without reuse and 8,493,012,480 with reuse: a negligible 2,048-byte difference. Q13 and Q18 were unchanged at 8,175,665,664 and 8,205,578,752. Q22 was 8,076,139,008 in both projection modes. These PyTorch peaks include the 8 GB pool, not all driver memory. Fewer temporary allocations do not necessarily reduce whole-query peak usage.

7. Correctness, tradeoffs and limitations

The SF1 test passed independently and with both switches enabled, but checks formatted leading rows rather than full outputs. Supplemental tests passed 110 complete aggregation-vector checks and 10 materialization-array checks; synthetic scaling also matched reference sums/means. Coverage includes threshold boundaries, skew, null counts, strided copies, integer/decimal/Boolean results and native large-integer precision. Empty inputs/groups and full SQL semantics remain unverified.

Reuse keeps an index live longer and offers little benefit outside its target path. Native projection retains the pool copy and does not help already-float64 results. Future work: full-output and empty-input tests, isolated conversion profiling, controlled idle-machine repetitions and larger TPC-H scales. Historical hash-collision and negative-value reduction behavior were not redesigned.

8. Exact reproduction: setup and source

Run on Linux x86_64 with the GPU driver already installed and Conda available. Do not change a shared driver. Obtain the submitted branch, check out verified code commit 045af6abf0f39fcc0ee8bd861b61005249842566, and place TQP-Inmem beside tpch-dbgen-tensors under one parent directory. The full instructions are also in REPRODUCE.md and linked from Setup.md.

```
# Place the supplied TQP-submission.bundle in the common parent directory.
git clone -b lyon/aggregation-materialization \
  TQP-submission.bundle TQP-Inmem
cd TQP-Inmem
git checkout 045af6abf0f39fcc0ee8bd861b61005249842566
conda create -y -n tqp-review python=3.12.14 pip -c conda-forge
conda activate tqp-review
python -m pip install torch==2.14.0 torchvision==0.29.0 \
  --index-url https://download.pytorch.org/whl/cu130
python -m pip install pandas==3.0.6 pyspark==4.2.0 \
  pyparsing==3.3.3 pytest==9.1.1
cd ..
git clone https://github.com/hfhongzy/tpch-dbgen-tensors.git
git -C tpch-dbgen-tensors checkout \
  83b538594b1af1e9a60b0fc9b715adf8cb09d23a
conda create -y -n tpch-data-review python=3.12 pip cmake=3.31 \
  ninja gcc_linux-64=11 gxx_linux-64=11 -c conda-forge
conda activate tpch-data-review
python -m pip install torch==2.6.0 \
  --index-url https://download.pytorch.org/whl/cpu
cd tpch-dbgen-tensors
mkdir -p data
cmake -S . -B build-cpu -DCMAKE_BUILD_TYPE=Release
cmake --build build-cpu --parallel 2
cd build-cpu
./dbgen -s 1
cd ../../TQP-Inmem
conda activate tqp-review
nvidia-smi -L
export CUDA_VISIBLE_DEVICES=GPU-47f261f4-a20b-afa9-e0ba-17bce50f44e7
mkdir -p results
```

The UUID above is for the tested machine; substitute the target GPU UUID from nvidia-smi -L elsewhere. The committed config.json points to ../tpch-dbgen-tensors/data relative to the TQP root. CPU generation avoids needing a CUDA compiler matched to GPU PyTorch. Generated SF1 tensor files occupy approximately 877 MB as reported by du on the original machine.

Verification status and environment capture

A separate detached Git worktree at the verified commit passed the 110 aggregation checks, 10 materialization checks, and combined SF1 test (1 passed, 73 warnings in 10.24 s). It reused existing data and the installed environment. A fully clean environment installation and regeneration were not repeated, so this is code-checkout verification, not a fresh-machine certification. Deprecation warnings remain.

```
python research_checks.py correctness
python materialization_checks.py
TQP_AGG_REUSE=1 TQP_PROJECT_NATIVE=1 python main.py --test
python --version
python -m pip freeze
conda list --explicit
nvidia-smi
lscpu
git rev-parse HEAD
```

Preserved evidence includes checkout test logs, the final four-way CSVs/logs, scaling and Q22 memory logs, pip freeze output, and the verified commit ID. Exact transitive versions should be taken from that capture; resolving a new environment can change them. Base TQP source: 934139d9c8abfe2dc982d64c2a390860133af90c.

9. Exact reproduction: runtime, profiling and memory

Execute this block in bash from the TQP root with the GPU environment active. Mode 00 is baseline, 10 aggregation only, 01 projection only, and 11 combined. Both switches default off. Each invocation uses SF1 and two whole-suite warmups.

```
task_final_dir=$(mktemp -d results/final-compare-XXXXXX)
for run in 1 2 3 4 5
do
  case "$run" in
    1|5) modes="00 10 01 11" ;;
    2) modes="10 01 11 00" ;;
    3) modes="01 11 00 10" ;;
    4) modes="11 00 10 01" ;;
  esac
  for mode in $modes
  do
    if ! TQP_AGG_REUSE=${mode%?} TQP_PROJECT_NATIVE=${mode#?} \
      python main.py --SF 1 --q 1 13 18 22 --warmup_iter 2 \
      --no_debug --time_output_file "$task_final_dir/mode-$mode.csv" \
      > "$task_final_dir/run-$run-mode-$mode.log" 2>&1
    then
      tail -n 20 "$task_final_dir/run-$run-mode-$mode.log"
      break 2
    fi
  done
done
```

For the earlier two-way aggregation comparison, use modes 00/10 and queries 1 13 18; for projection, use 00/01 and queries 1 22, in that fixed order for five rounds. Compare medians and preserve ranges/raw observations rather than selecting only favorable runs.

```
python aggregation_scaling_benchmark.py
TQP_AGG_REUSE=0 TQP_PROJECT_NATIVE=0 python research_checks.py memory
TQP_AGG_REUSE=1 TQP_PROJECT_NATIVE=0 python research_checks.py memory

# Q22 peak memory: the exact loop used in the original experiment.
python - <<'PY'
import os, gc, torch, main
os.environ["TQP_AGG_REUSE"] = "0"
records = []
with torch.inference_mode():
    for mode in ("0", "1"):
        os.environ["TQP_PROJECT_NATIVE"] = mode
        main.warmup()
        main.warmup()
        for run in range(1, 4):
            gc.collect()
            torch.cuda.synchronize()
            torch.cuda.empty_cache()
            torch.cuda.reset_peak_memory_stats()
            main.exec(1, [22], None, None, True, None, None, None)
            torch.cuda.synchronize()
            records.append((mode, run, torch.cuda.max_memory_allocated()))
print("mode,run,peak_allocated_bytes")
for record in records:
    print(",".join(map(str, record)))
PY

TQP_AGG_REUSE=0 TQP_PROJECT_NATIVE=0 python main.py --SF 1 --q 1 \
  --no_debug --torch_profile=results/profile-original
TQP_AGG_REUSE=1 TQP_PROJECT_NATIVE=0 python main.py --SF 1 --q 1 \
  --no_debug --torch_profile=results/profile-reuse
```

Profiling runs intentionally differ from warmed timing runs and must not supply headline speedups. Profiler allocation totals are not peak memory. The scripts report PyTorch allocator measurements; allocator reservation and total device usage are distinct. Record concurrent GPU activity in future reruns.